

Oro AP Appraisal App: Technical Handover and Database Integration Specification

Part 1 — Executive Summary

Part 2 — Current Data Model (Source: Firebase Firestore)

Part 3 — Migration Target: What “Integration with Oro’s Core Database” Actually Requires Deciding

Part 4 — Data Migration Logic: Field-by-Field Transformation Rules

Part 5 — Business Logic That Must Be Re-Implemented, Not Just “Migrated”

Part 6 — Open Decisions Requiring Your Team’s Input

Part 7 — What’s Included in This Handover Package

Prepared for: whichever engineering team takes on integrating this app with Oro's core database Prepared by: Oro Corp (V), sole developer to date; technical continuity: Vivek, from end of July 2026) Purpose: a complete, self-contained technical brief covering what this app does, exactly what data it holds today, what a database integration would require, and every piece of business logic that must survive any such change. Written so an engineering team can plan and scope the integration without needing a live walkthrough for the basics.

Part 1 — Executive Summary

The Oro AP Appraisal App is actually two related but independent browser-based tools used by Oro's Lending Services Audit team:

- AP Appraisal Calibration — a quarterly gold-purity test administered to each Appraisal Partner (AP): the AP appraises 5 "needles" of known purity, an auditor records what the AP reported, and the app scores each answer against a fixed SOP, computes a Pass/Fail, and stores the record. Replaces what used to be a manual Google Sheet process.
- AP Interview — tracks candidate interview results during AP hiring. A simpler record-keeping tool: no scoring formula, no auto-fail rule, no external data pulled in beyond what an interviewer types.

They are presented to the user as two selectable "workspaces" from a shared landing page, but are genuinely independent apps under the hood — separate Firestore collections, separate entry forms, separate dashboards, separate Excel export formats. A small amount of shared plumbing (auth, city list, date-range math, a couple of UI helpers) is common code, called out explicitly in Part 2.

Current architecture: static HTML/CSS/JS frontend (no framework, no build step) + Firebase (Firestore for app data, Firebase Authentication for login) + Vercel (hosts the frontend and two serverless API functions). The app's only existing external data connection is a one-way sync from a Google Sheet (Oro's AP master roster) into Firestore — the app has never read from or written to any system describable as "Oro's core database."

What's changing: this document exists because that is about to change — this app needs to be integrated with Oro's core database. Unlike a typical migration brief, **the target system here is not yet specified**. Part 3 lays out what needs to be established before an actual migration plan can be written, in the same shape a migration-target decision would normally take (this document deliberately mirrors the structure of a comparable handover done for a sibling Oro project, the Tenmark Audit App, whose migration target — PostgreSQL vs. MongoDB on Tenmark's own infrastructure — was already confirmed at the time that document was written; no equivalent confirmation exists yet here).

This document focuses on making Part 2 (the current data model) and Part 5 (the business logic that must survive any change) as concrete and complete as possible, since those are gatherable today regardless of what the target system turns out to be. Part 3 and Part 6 cover what is genuinely undecided and needs input from whoever owns Oro's core database before further planning can proceed.

Part 2 — Current Data Model (Source: Firebase Firestore)

Firestore is a NoSQL document database — there is no schema enforcement, no foreign keys, no joins. Every fact below was extracted directly from reading the actual application code (`js/scoring.js` , `js/interviewStats.js` , `firestore.rules` , the two serverless functions), not from assumption or memory. Six collections/documents exist.

2.1 Collection: `calibrations`

One document = one AP's one quarterly calibration test. There is no separate "current state per AP" concept — every quarter's test is its own permanent record, and the app has no update/delete path at all (enforced by `firestore.rules` , not just convention).

Field	Type (as stored)	Nullable	Description
<code>city</code>	string	No	Must exactly match the spelling used in the AP roster Google Sheet's Location column — see Part 4.
<code>testDate</code>	string, YYYY-MM-DD	No	Built from Day/Month/Year dropdowns on the entry form, not a native Firestore Timestamp.
<code>monthYearLabel</code>	string, e.g. "Mar 2024"	No	Display/quarter-grouping convenience, derived from <code>testDate</code> at write time — not re-derived on read.
<code>auditorName</code>	string	No	Selected from the shared <code>config/auditors</code> roster (see 2.4) — stored as a copy, not a reference.
<code>auditorEmpCode</code>	string	No	Copy of the auditor's employee code at time of entry.
<code>apName</code>	string	No	The Appraisal Partner being tested. Autofilled from <code>appraisal_partners</code> (2.3) by Employee Code, but remains editable and is stored as a plain copy.
<code>apEmpCode</code>	string	No	AP's employee code.
<code>apDoj</code>	string, YYYY-MM-DD	No	AP's date of joining, autofilled same as above.
<code>needles</code>	array of exactly 5 objects	No	{ given: string, answer: string, score: number } per needle — see Part 5.1 for the scoring rule that produces <code>score</code> . Enforced to be exactly 5 entries by <code>firestore.rules</code> .
<code>totalScore</code>	number, 0-10	No	Sum of the 5 needles' <code>score</code> values. Auto-computed client-side; not independently verifiable server-side beyond the 0-10 range check in the rules.
<code>autoFailTriggered</code>	boolean	No	True if any single needle hit the SOP's hard auto-fail condition — see Part 5.1.
<code>result</code>	string, enum "Pass" "Fail"	No	Enforced by <code>firestore.rules</code> .
<code>remarks</code>	string	Yes	Optional free text.
<code>createdAt</code>	Firestore server timestamp	No	Write-time timestamp.

Known gap, flagged rather than papered over: there is no `appraisal_partners` -style master record referenced by ID inside `calibrations` itself — `apName` / `apEmpCode` / `apDoj` are stored as a snapshot copy on every record, not a live reference. A later change to an AP's name in the roster will not retroactively update historical calibration records, by design (this

preserves what was actually recorded at test time) but worth knowing if any downstream reporting assumes otherwise.

2.2 Collection: `interview_entries`

One document = one candidate's one interview record. Same no-update/no-delete rule as `calibrations`.

Field	Type (as stored)	Nullable	Description
<code>city</code>	string	No	Same shared city list as calibration.
<code>interviewDate</code>	string, YYYY-MM-DD	No	
<code>candidateName</code>	string, non-empty (enforced)	No	
<code>locationDetail</code>	string	Yes	Optional; used when the candidate's actual location differs from the <code>city</code> field (e.g. a specific branch).
<code>company</code>	string	No	Candidate's current/previous employer.
<code>role</code>	string	No	
<code>age</code>	number	No	
<code>experience</code>	string	No	Free text, e.g. "2 Years" — not a number.
<code>bikeAvailable</code>	string, enum "Yes" "No"	No	Enforced by <code>firestore.rules</code> .
<code>dlAvailable</code>	string, enum "Yes - DL" "Yes - LLR" "No"	No	Enforced by <code>firestore.rules</code> .
<code>scoreTheory</code>	number, 0-4	No	Enforced range.
<code>scorePractical</code>	number, 0-6	No	Enforced range.
<code>totalScore</code>	number, 0-10	No	Manually typed by the interviewer, not auto-summed — unlike calibration's <code>totalScore</code> . If any migration or reporting logic assumes this is always <code>scoreTheory</code> + <code>scorePractical</code> , verify that assumption against real data first; it will not hold in every record.
<code>localLanguage</code>	string	No	Derived from <code>city</code> via a lookup table (<code>config/cityLanguages</code> , 2.5), not typed by the interviewer.
<code>localLanguageProficiency</code>	string	No	
<code>englishProficiency</code>	string	No	
<code>round1Decision</code>	string	No	See the note immediately below — this field has a real enforcement/legacy-data split that any migration must handle correctly.
<code>interviewer</code>	string	No	Pulled from the same shared roster as calibration's auditors (<code>config/auditors</code> , 2.4) — there is no separate interviewer list.
<code>remarks</code>	string	Yes	
<code>createdAt</code>	Firestore server timestamp	No	

Migration note — `round1Decision`, mandatory reading before touching this field: `firestore.rules` currently constrains every *new* write of this field to exactly "Selected" or "Rejected" (enforced via a dropdown on the entry form). However, roughly 159 historical records imported before this constraint existed still contain free-text values such as "Rejected due to lack of appraisal knowledge", "Not Selected", or "Ok for the next level". A function called `classifyDecision()` (in

`js/interviewStats.js`) exists specifically to correctly bucket both old and new values into Selected/Rejected/Other for reporting — see Part 5.2. Any migration script or new reporting logic that reads this field via a plain string comparison instead of this classification logic will silently misclassify a real, non-trivial slice of historical records. This is not a hypothetical risk — it already caused one production bug (a city dashboard showing 0% “Selected” due to an unrecognized legacy phrasing) before being caught and fixed.

2.3 Collection: `appraisal_partners`

Document ID = AP’s Employee Code (e.g. `0R000020`). Read-only from the app’s perspective — populated by a one-way sync described in Part 4.4, not written to by any app code.

Field	Type	Nullable	Description
<code>name</code>	string	No	
<code>empCode</code>	string	No	Duplicate of the document ID, also stored as a field.
<code>doj</code>	string, <code>YYYY-MM-DD</code>	No	

The source Google Sheet may carry additional columns beyond these three — the app only reads what it needs for autofill; do not assume this is the complete set of fields Oro’s roster actually tracks for an AP.

2.4 Document: `config/auditors`

Single document, field `list`: array of `{ name: string, empCode: string, email?: string }`. Shared between both apps — used for calibration’s Auditor field and interview’s Round 1 Interviewer field. `email` is only present on entries created via the `api/create-user.js` endpoint (2.6); older entries added directly through the Settings UI may lack it.

2.5 Document: `config/cityLanguages`

Single document, field `map`: a plain map of city name → language string (interview-only; not a calibration concept). Has a hardcoded fallback table in `js/cityLanguages.js` for the original 8 cities if this document doesn’t exist, so its absence does not break the app — worth knowing if a migration script checks for “is this document present” as a completeness signal; its absence is a valid, handled state, not necessarily an error.

2.6 Documents: `config/cities` and `config/managers`

- `config/cities` — field `list`, array of city name strings. Shared city list for both apps. **Spelling must exactly match the AP roster Sheet’s Location column** — a mismatch silently produces a city with no data rather than an error.
- `config/managers` — field `emails`, array of strings, **must be lowercase**. The only authorization distinction in either app: Manager (Settings access + write access to the config documents above) vs. everyone else. Not editable from the app itself — Console-only by design. There is no separate auditor/interviewer *role* — any authenticated `@orocorp.in` account can use either app’s main features regardless of Manager status.

2.7 Backend endpoints touching this data (not Firestore collections themselves, but part of the data model’s write path)

- `api/create-user.js` — creates a Firebase Auth login account and appends to `config/auditors.list` in one request.
- `api/remove-auditor.js` — removes from `config/auditors.list` and disables the corresponding Firebase Auth account.

Both authorize every action using the calling Manager’s own ID token (verified server-side), never a client-supplied identity claim — see `lib/firebaseAdminHelpers.js` for the shared verification logic.

Part 3 — Migration Target: What “Integration with Oro’s Core Database” Actually Requires Deciding

This is the section a comparable handover document would normally use to lay out a confirmed target system’s tradeoffs (for example: JSONB column vs. normalized table vs. embedded document, for a team that has already confirmed which database engine they’re moving to). **That confirmation does not yet exist for this app.** Rather than skip this section or fabricate a recommendation for an unspecified target, it’s written here as what needs to be established first, in the same shape that decision will eventually take.

3.1 The core tension, once a target is known, will very likely resemble this app’s sibling migration

If “Oro’s core database” turns out to be a relational system (Postgres-family or similar), the same structural question that came up for the Tenmark Audit App’s migration will come up here too: `needles[]` (calibration) and, to a lesser extent, the flat-but-numerous fields on `interview_entries`, are natural fits for a document store but require a real decision in a relational target:

- Option A — a JSONB/equivalent column holding `needles` exactly as it’s shaped today. Lowest-risk, no data restructuring, but no SQL-level querying into individual needle results (e.g. “find every calibration where needle 3 specifically triggered an auto-fail” would require JSON query operators, not a plain column comparison).
- Option B — a fully normalized `needle_results` table with a foreign key back to `calibrations`, one row per needle (5 rows per calibration record). Enables real per-needle SQL queries, at the cost of a genuinely larger migration script and schema design effort.
- Option C — if the target turns out to be a document database (MongoDB or similar) instead, `needles` as an embedded array is close to a direct structural match for what exists today.

No recommendation is made here between these, because unlike the Tenmark case, **it is not yet known which database engine, if any, this integration is targeting.** Whoever picks this up should treat this exact tradeoff table as the starting point for that conversation once the target is confirmed, rather than re-deriving it from scratch.

3.2 What is different here versus a typical “we know the target” migration brief

- No draft schema files exist yet for any specific target (contrast with a situation where Sequelize models and Mongoose schemas have already been drafted and validated against a confirmed stack) — because there is currently nothing to draft against.
- No confirmation exists of read vs. write direction: does this app need to read AP/candidate identity data out of Oro’s core database (replacing the current Google Sheet sync, Part 4.4), write its own results into it, or both?
- No confirmation exists of network reachability: this app’s current backend is two stateless Vercel serverless functions calling Firebase’s REST API over the public internet. If Oro’s core database requires VPN/private networking, that is new infrastructure for this app, not a code change to existing infrastructure.

3.3 What must be preserved regardless of what target and shape are eventually chosen

1. `calibrations` and `interview_entries` must remain append-only in spirit — there is currently no update/delete path at all, enforced at the database layer. If the new system allows updates, a deliberate decision is needed about whether to preserve that audit-integrity guarantee (e.g. via database permissions, application-layer restriction, or an explicit versioning/history table) rather than silently losing it because the new database technically permits editing rows.
 2. `needles[].score` and `totalScore / autoFailTriggered / result` must remain derivable from (or verifiably consistent with) the SOP scoring rule in Part 5.1 — this is business logic, not a storage detail, and no column type in any target database will enforce it automatically.
 3. `round1Decision`’s legacy-vs-enum split (Part 2.2) must be handled with the same care as any currency-string-style data-cleaning problem — see Part 4.3 for the exact handling required, directly analogous to how a formatted-string numeric field would need cleaning during any migration.
-

Part 4 — Data Migration Logic: Field-by-Field Transformation Rules

This section specifies what needs to happen to correctly move data out of Firestore, regardless of what it moves into.

4.1 Extraction

Firestore data can be exported via Google Cloud's Firestore export tool (produces files in a GCS bucket) or read directly via the Firestore REST API / Admin SDK, paginated. Given the dataset size — roughly 789 calibration records and 159 interview records as of this handover, both well under a thousand — a direct read-and-transform script is entirely practical; there is no need for bulk export tooling at this scale.

4.2 Document ID handling

Firestore's auto-generated document IDs (for `calibrations` and `interview_entries`) are opaque, non-sequential strings. Decide on a new primary key strategy for whatever target is chosen: - If UUID: generate a fresh UUID per record at migration time — do not attempt to reuse or reformat the Firestore ID. - If auto-increment integer: likewise generate fresh sequential IDs; the old Firestore ID becomes irrelevant after migration (optionally preserve it in a `legacyFirestoreId` column for one-time debugging during the transition, droppable afterward).

`appraisal_partners` is the one collection where the existing document ID (Employee Code) is already a meaningful business key, not an opaque ID — this one is a candidate for reuse as a natural key in a new system, unlike the other two collections' IDs.

4.3 The `round1Decision` legacy-value handling step — mandatory, not optional

This is the single most important data-cleaning step in this migration, directly analogous to a formatted-currency-string cleanup problem in a numeric field: silently getting this wrong doesn't crash anything, it just produces confidently wrong numbers in every future report.

The problem: `firestore.rules` enforces `round1Decision` in `['Selected', 'Rejected']` for every write today, but this constraint was added after roughly 159 historical records already existed with inconsistent free-text values (e.g. "Rejected due to lack of appraisal knowledge", "Not Selected", "Ok for the next level").

The required transformation, to be applied to every historical record during migration: 1. If the raw value is already exactly "Selected" or "Rejected" → use as-is. 2. Otherwise, run it through the same classification logic as `classifyDecision()` in `js/interviewStats.js`: check for "not selected" or "reject" (case-insensitive substring) first → classify as Rejected; else check for "select" or a "next level" phrasing (excluding anything containing "not ok") → classify as Selected; else → classify as "Other". 3. **Decide, as part of the migration plan, whether the target system's `round1Decision`-equivalent column should store the classified value ("Selected" / "Rejected" / "Other"), the original raw text, or both** (e.g. a clean enum column plus a `rawDecisionText` column preserving the original for audit purposes). This document does not make that call — it depends entirely on what the receiving reporting/dashboard logic needs, which is unspecified until Part 6's open questions are answered.

Validation step, mandatory: after transformation, spot-check a sample of records against their original raw text before trusting a full migration run — a legacy phrasing this pattern-matching logic doesn't anticipate could silently produce an "Other" classification for a record that a human would clearly call Selected or Rejected. This is the same category of risk as a currency-string regex missing an unexpected symbol format.

4.4 The AP-roster sync — not a Firestore field, but essential context for `appraisal_partners`

`appraisal_partners` is populated entirely by a one-way sync: a Google Apps Script bound to an external Google Sheet (Oro's AP master roster, "AP Details" tab) pushes changes into this Firestore collection on every Sheet edit. The app never writes to this collection itself. If any future integration intends to source AP identity data directly from Oro's core database instead of this Sheet-based pipeline, that is a **replacement** of this sync mechanism, not an extension of it — and whoever builds that replacement needs to decide whether the Sheet-editing workflow that Oro's HR/roster team currently uses continues to exist alongside the new pipeline, or is retired in favor of editing the core database directly.

4.5 Timestamps and date handling

All date fields (`testDate` , `interviewDate` , `apDoj`) are stored as plain "YYYY-MM-DD" strings, not native Firestore Timestamps — chosen deliberately so simple string comparison sorts identically to true date order. `createdAt` is the one field that is a native Firestore server timestamp. When migrating, confirm whether the target system's equivalent "creation time" field should be a genuine timestamp type (recommended) or preserved as the original UTC value — no timezone-conversion bug has been identified in this app's own date-string fields (unlike a comparable issue found in a

sibling Oro project's UTC-vs-IST daily-counter logic), but this has not been exhaustively audited against every possible edge case (e.g. a record entered right at a day boundary in a browser set to a non-IST timezone) — worth a deliberate check during migration rather than an assumption either way.

4.6 What does NOT need migrating

- The `source: "metabase-sync"`-equivalent concept does not exist in this app at all — unlike a sibling Oro project, there is no placeholder-record mechanism here to decide about.
 - `config/managers` and `config/cityLanguages`'s hardcoded-fallback behavior are operational/authorization configuration, not audit data — whether these need to exist in a new system at all depends entirely on Part 6's authentication-consolidation question, not on anything specific to this data model.
-

Part 5 — Business Logic That Must Be Re-Implemented, Not Just “Migrated”

A database migration moves data. It does not, by itself, move the rules that keep that data meaningful. The following logic currently lives in application code and must be deliberately reimplemented in whatever new application layer reads/writes this data — none of it is enforced by the database schema itself, in either the old or a new system.

5.1 Calibration scoring rule (`js/scoring.js`) — pure functions, no Firestore/DOM dependency, safe to port directly

Per needle, comparing the known karat (`given`) against what the AP reported (`answer`):

Condition	Score	Auto-fail?
Exact match	2	No
Off by 1 karat (both normal karats: 22K/21K/20K/19K/18K)	1	No
Off by 2+ karats (both normal karats)	0	No
Known = “Below 18K”, AP answers a normal karat	0	Yes — AP missed a genuinely sub-18K item
AP answers “Below 18K”, known is a normal karat	0	No — false alarm, not a missed detection

Overall: `totalScore` = sum of the 5 needle scores (max 10). **Pass** requires `totalScore >= 7` AND no auto-fail triggered by any needle. **Fail** if `totalScore <= 6`, OR if any single needle triggered auto-fail — regardless of total score. The auto-fail rule is a hard stop; it cannot be overridden by an otherwise-high score. This is directly analogous to a “deliberately conservative, do not improve this” rule in a sibling Oro project’s re-audit matching logic — the auto-fail behavior is a considered SOP decision, not an implementation gap to smooth over during a rewrite.

5.2 Interview decision classification (`js/interviewStats.js` → `classifyDecision()`)

Already specified in full in Part 4.3, since it doubles as both a migration-time data-cleaning step and an ongoing piece of business logic that any new dashboard/report must keep calling (never replace with a plain equality check against the raw field) for as long as any pre-enum legacy data exists in the system.

5.3 Read-cost-aware aggregate calculations

The all-cities dashboards (Total/Pass/Fail for calibration; Total/Selected/Rejected/Selection Rate for interview) and both all-cities Excel reports are, as of this handover, implemented using either Firestore count-aggregation queries (calibration dashboard, since `result` is a strict enum and an equality-based count is provably exact) or server-side date-range-narrowed document fetches (interview dashboard and both report pages, since `round1Decision`’s legacy-data classification genuinely requires reading real documents, not just counts). Whatever replaces Firestore should replicate the *intent* of this design — aggregate counts computed via the target database’s own native aggregation capability wherever the underlying field is a strict enum, falling back to fetching real rows only where classification logic that can’t be expressed as a database-level equality check is required. Do not default to “just fetch everything and count in application code” for the calibration side specifically — that was the exact pattern this app moved away from, for a documented cost reason (Firestore’s free-tier daily read quota).

5.4 Access control model (current state, pending your team’s decision)

Enforced today via Firestore Security Rules (`firestore.rules` , included in this package), not application code — meaning these rules are enforced at the database layer, not just the UI: - Any authenticated `@orocorp.in` session can read `calibrations` , `interview_entries` , `appraisal_partners` , and all `config/*` documents. - Any authenticated `@orocorp.in` session can *create* (never update or delete) `calibrations` / `interview_entries` records, subject to the field-presence and type/enum checks specified in Part 2.1/2.2. - Only a verified Manager (`config/managers.emails`) can write to `config/cities` , `config/auditors` , or `config/cityLanguages` . - `config/managers` itself cannot be written from the app at all under any role — Console-only. - `appraisal_partners` cannot be written from the app at all — sync-only, via the mechanism in Part 4.4.

Whatever database and backend eventually replace or sit alongside Firestore must implement equivalent access control at the application/API layer, since neither a typical relational nor document database has a native equivalent to Firestore Security Rules — this logic cannot simply be “turned off” without a replacement; it must be deliberately rebuilt, most likely as backend middleware of whatever framework the receiving system uses.

Part 6 — Open Decisions Requiring Your Team’s Input

These are not gaps in this document — they are genuine open questions that only whoever owns Oro’s core database can answer, listed here so nothing proceeds on an unconfirmed assumption:

1. **What “Oro’s core database” actually is** — engine, hosting, ownership, and whether it’s the same system referenced elsewhere in Oro’s engineering org as the lending/loan database.
2. **Direction of integration** — does this app need to read AP/candidate identity data out of that system (replacing the current Google Sheet sync, Part 4.4), write its calibration/interview results into it, or both?
3. **Network/access model** — is Oro’s core database reachable from this app’s current stateless-serverless-function architecture, or does it require private networking this app has no existing path to?
4. **Authentication consolidation** — does this integration also imply replacing this app’s standalone Firebase Authentication with whatever login system Oro’s core database’s other consumers already use? This single decision affects whether `config/managers` /the Manager-authorization model need to be rebuilt against a different identity system entirely, not just have their storage moved.
5. **Schema mapping** — how does this app’s `apEmpCode` /city-name-string convention correspond to whatever identifiers Oro’s core database uses for the same real-world entities? There is no existing confirmation that these match without checking.
6. **Rate limits / cost model** for whatever access mechanism is chosen, given this app currently runs on Firebase’s free Spark tier with hard daily quota ceilings — whether an equivalent ceiling exists on the core-database side is unknown here.
7. **Operational ownership** of the resulting integration once built — whether the core-database team and this app’s ongoing maintainer (transferring to Vivek) are the same team, a different team, or require a formal request process for any future change.

Until 1–4 above are answered, no concrete migration schema or transformation script can responsibly be written — doing so before those answers exist would mean guessing at a target, the same mistake this document deliberately avoids in Part 3.

Part 7 — What’s Included in This Handover Package

File/Folder	Contents
README.md	The project’s own README — architecture, data model, deployment process, and a “known gaps” section.
TECH-TRANSFER-0ro-AP-Appraisal-App.md	The broader technical handover document this specification’s Part 2 and Part 5 were built from — covers deployment, Firebase project configuration, the AP-roster sync’s operational risk, and recent incident history not repeated here.
firestore.rules	The current database-layer access control rules referenced in Part 5.4.
vercel.json	Deployment configuration (clean URLs, security headers).
js/scoring.js	Pure-function calibration scoring logic (Part 5.1) — safe to port with minimal adaptation, since it has no Firestore or DOM dependency.
js/interviewStats.js	classifyDecision() and related logic (Parts 4.3, 5.2).
js/dateRangeUtils.js	Shared date-range filtering math, referenced in Part 5.3.
api/create-user.js , api/remove-auditor.js , lib/firebaseAdminHelpers.js	The two backend endpoints and shared helper referenced in Part 2.7 and Part 5.4.
The full application source (all *.html , js/*.js , css/styles.css)	The actual running application, for direct reference alongside this document.

Not included, because it does not yet exist: any draft schema file, migration script, or transformation code for a specific target database — see Part 3.2 for why, and Part 6 for what must be answered before that work can responsibly begin.

Recommended reading order for your engineering team: this document first (for the integration-specific plan and open questions), then TECH-TRANSFER-0ro-AP-Appraisal-App.md for broader operational context (deployment workflow, the AP-roster sync’s single-point-of-failure risk, recent incident history), then README.md and firestore.rules as detailed references while actually scoping integration work.